



Satish Hattekar <satishhattekar1997@gmail.com>

B28: SQL SuperHero Program : Session 1

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

9 February 2025 at 00:42

Hi All,

Please find the below notes from all what we discussed in our today's meeting. Notes will be provided after every class.

For live sql practice, if you do not have tool:

<https://livesql.oracle.com/apex/f?p=590:1:28733181935303::NO::>

<Create or account here and use this free tool with latest oracle version available here>

=====

Tables are basic fundamental storage structure of databases. table which stores data in it. They stored data in rows and columns. Mainly 4 types of tables:

1. Heap Organized
2. Index Organized
3. External Tables
4. Temporary Tables

by default all tables in oracle are heap organized tables. It means oracle inserts a new row, when it gets new data. a new row inserted every time.

Index organized means, everything go on correct place. insert may took costly but selection is easy.

Why Use Index Organized Tables

Accessing data via the primary key is quicker as the key and the data reside in the same structure. There is no need to read an index then read the table data in a separate structure.

Lack of duplication of the key columns in an index and table mean the total storage requirements are reduced.

An index-organized table (IOT) is a type of table that stores data in a B*Tree index structure.

Normal relational tables, called heap-organized tables, store rows in any order (unsorted).

Creation Of Index Organized Tables

To create an index organized table you must:

- Specify the primary key using a column or table constraint.
- Use the ORGANIZATION INDEX.

Advantage:

An index-organized table keeps its data sorted according to the primary key column values for the table. An index-organized table stores its data as if the entire table was stored in an index. Indexes serve two main purposes:

- To enforce uniqueness When a PRIMARY KEY or UNIQUE constraint is created, Oracle creates an index to enforce the uniqueness of the indexed columns.
- To improve performance When a query can use an index, query performance may dramatically improve.

Disadvantage:

- You must have a primary key on the table with a unique value.
- You cannot have any other indexes on the data.
- You cannot partition an index-organized table.

The screenshot shows the SQL Developer interface with the Query Builder tab active. The SQL statement is:

```
CREATE TABLE locations
(
  id NUMBER(10) Primary Key,
  description VARCHAR2(50),
  map varchar2(10)
)
ORGANIZATION INDEX;
```

The **ORGANIZATION INDEX;** clause is circled in red. A handwritten red note with an arrow points to it, saying "keyword to create index organized table". Below the query, the "Script Output" pane shows "Table LOCATIONS created." and "Task completed in 0.027 seconds".

An index-organized table (IOT) is a type of table that stores data in a B*Tree index structure. Normal relational tables, called heap-organized tables, store rows in any order (unsorted). In contrast to this, index-organized tables store rows in a B-tree index structure that is logically sorted in primary key order.

--> For Creating an IOT(index-organized table), you must need a PK. without that it gives error.

The screenshot shows the SQL Developer interface with the Query Builder tab active. The SQL statement is:

```
1 create table location (
2 id number(10), -- primary key,
3 description varchar2(10)
4 ) organization index
```

Handwritten green notes with arrows point to the statement: "Without PK" points to the comment on line 2, and "Trying to create IOT table" points to the **organization index** clause on line 4.

Below the query, the "Script Output" pane shows an error:

```
Error starting at line : 1 in command -
create table location (
id number(10), -- primary key,
description varchar2(10)
) organization index
Error report -
ORA-25175: no PRIMARY KEY constraint found
25175. 00000 - "no PRIMARY KEY constraint found"
*Cause: A PRIMARY KEY constraint must be defined for a table
with this organization
*Action: Define a PRIMARY KEY
```

A handwritten green arrow points from the word "error" to the error message.

data Dictionary

```
select * from user_tables
```

ORGANIZED	PARTITIONED	IOT_TYPE	TEMPORARY
	NO	IOT ✓	N
	NO	-	N



If we check the data dictionary (User_tables) and see the column IOT_TYPE , so we can find the IOT table have the value IOT. Which identifies that this table is IOT table.

--> By default IOT tables are sorted. See the easy example of these both tables as below:

-- Insert in Heap organized table (Normal tables)

```
insert into test values (1,'one');
insert into test values (3,'three');
insert into test values (4,'four');
insert into test values (2,'two');
```

-- Insert in Index organized table (IOT tables)

```
insert into location values (1,'one');
insert into location values (3,'three');
insert into location values (4,'four');
insert into location values (2,'two');
```

```
select * from test
```

By default
not
sorted

ID	DESCRIPTION
1	one
3	three
4	four
2	two

4 rows selected.

```
select * from location
```

Data
stored
in
sorted
way

ID	DESCRIPTION
1	one
2	two
3	three
4	four

4 rows selected.

Types of SQL Statement :

- **DDL** --> create, alter, drop, truncate
 - No commit or rollback . It is autocommit.
 - DDL statements are used to build and modify the structure of your tables and other objects in the database. When you execute a DDL statement, it takes effect immediately.
- **DML** --> Insert, update, delete
 - DML statements are used to work with the data in tables.
- **TCL** --> commit, rollback, savepoint

COMMIT: Commit command is used to permanently save any transaction into the database.

ROLLBACK: This command restores the database to last committed state. It is also used with savepoint command to jump to a savepoint in a transaction.

SAVEPOINT: Savepoint command is used to temporarily save a transaction so that you can rollback to that point whenever necessary.

```
select * from mycity;
```

Query Result	
All Rows Fetched: 4 in 0.003 s	
ID	NAME
1	UK
2	Mumbai
3	Chennai
4	Kolkatta

```
update mycity set name = 'Noida' where id = 1;
savepoint A;
update mycity set name = 'Noida' where id = 3;
savepoint B;
```

2 save point created

2 rows updated

Query Result	
Task completed in 0.019 seconds	
1 row updated.	
Savepoint created.	
1 row updated.	
Savepoint created.	

Now, I can go and revert back until to a particular point and rest was still unchanged.

The screenshot shows a SQL script editor with the following commands:

```

update mycity set name = 'Noida' where id = 1;
savepoint A;
update mycity set name = 'Noida' where id = 3;
savepoint B;
rollback to savepoint A;
select * from mycity
  
```

Handwritten annotations include:

- An orange arrow pointing from the second `update` statement to the `rollback to savepoint A;` statement, with the text "Till here rpl/rollback" written in orange.
- A green arrow pointing from the word "still" to the first row of the query result, with the word "unchanged" written in green.
- A purple arrow pointing from the word "Back to chennai" to the third row of the query result.

The query result table is as follows:

ID	NAME
1	1 Noida
2	2 Mumbai
3	3 Chennai
4	4 Kolkatta

Difference between DDL and DML:

DDL	DML
It stands for Data Definition Language.	It stands for Data Manipulation Language.
It is used to create database schema and can be used to define some constraints as well.	It is used to add, retrieve or update the data.
It basically defines the column (Attributes) of the table.	It add or update the row of the table. These rows are called as tuple.
It doesn't have any further classification.	It is further classified into Procedural and Non-Procedural DML.
Basic command present in DDL are CREATE, DROP, RENAME, ALTER etc.	BASIC command present in DML are UPDATE, INSERT, MERGE etc.
DDL does not use WHERE clause in its statement.	While DML uses WHERE clause in its statement.

Question: I am altering a table and adding a column. Now I want to remove it. Can I do rollback?
 No, DDL is autocommit, not required commit or rollback

One more question on DDL command:

I have a table Test and that table have id column.

Step 1: Inserted records with id = 1

Step 2: Inserted records with id = 2
Step 3: Create Index on another table Test2
Step 4: Rollback;

How many records in table test as we did the rollback.

Answer: 2 records as we inserted on Step 1 and step 2. because on Step 3 we fired DDL and this is the property of a DDL command to do auto commit of everything what is happened before that DDL.

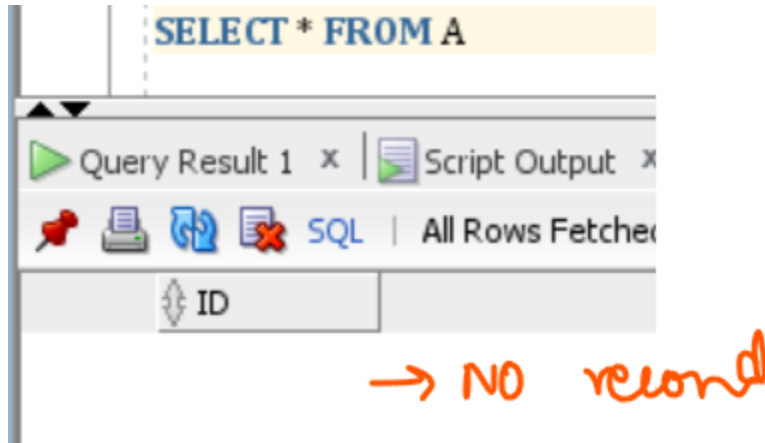
CREATE TABLE A (ID NUMBER) ; -- New table created with 0 records

Scenarion 1:

```
/* 4 rows Inserted */  
INSERT INTO A VALUES (1);  
INSERT INTO A VALUES (2);  
INSERT INTO A VALUES (3);  
INSERT INTO A VALUES (4);
```

Rollback;

Rollback issues, means everything reverted back.



Scenarion 2:

```
/* 4 rows Inserted */  
INSERT INTO A VALUES (1);  
INSERT INTO A VALUES (2);  
INSERT INTO A VALUES (3);  
INSERT INTO A VALUES (4);
```

Commit;

Rollback;

Once something is committed, means committed, you can not rollback. Here there is no impact of rollback.

```
commit;  
ROLLBACK; → No impact  
SELECT * FROM A
```

Query Result 1 x | Script Output x | Query Result

SQL | All Rows Fetched: 4 in 0.003 seconds

ID
1
2
3
4

4 records

Scenario 3:

/* 4 rows Inserted */

```
INSERT INTO A VALUES (1);  
INSERT INTO A VALUES (2);  
INSERT INTO A VALUES (3);  
INSERT INTO A VALUES (4);
```

After that I issued a DDL command. DDL can be anything, alter, truncate, drop, create sequence, create index anything

```
CREATE INDEX IDX_T on B(ID);  
ROLLBACK; → Auto commit  
SELECT * FROM A → No impact
```

Query Result 1 x | Script Output x | Query Result 2 x | Query Result 3 x

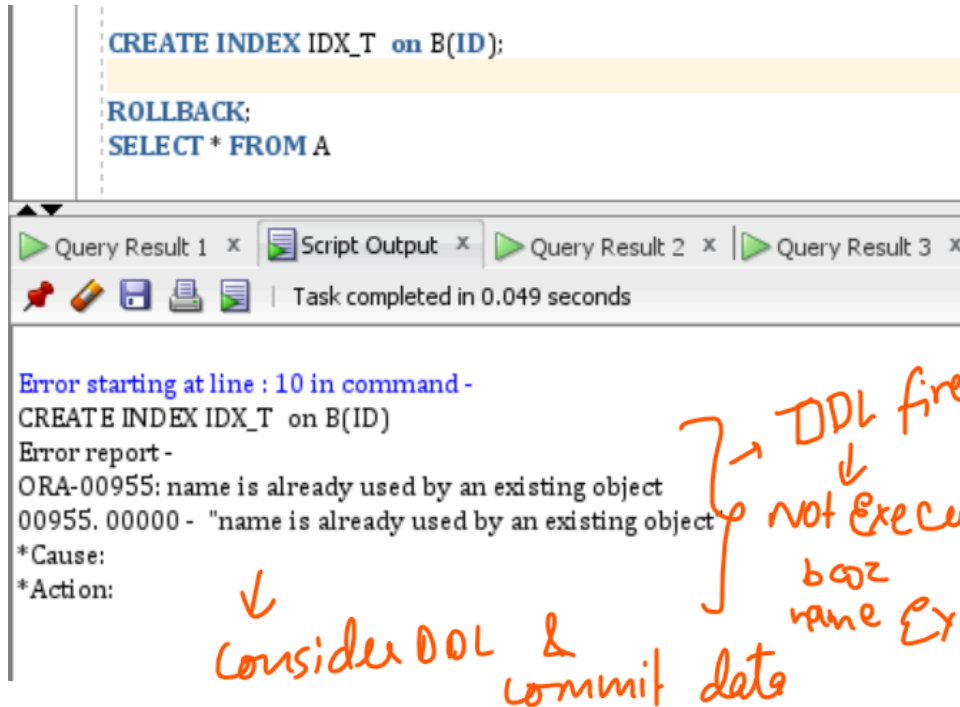
SQL | All Rows Fetched: 4 in 0.002 seconds

ID
1
2
3
4

Scenario 4:

```
/* 4 rows Inserted */  
INSERT INTO A VALUES (1);  
INSERT INTO A VALUES (2);  
INSERT INTO A VALUES (3);  
INSERT INTO A VALUES (4);
```

After that I issued a DDL command. but my ddl fail.



The screenshot shows a database client window with a script editor at the top containing the following SQL commands:

```
CREATE INDEX IDX_T on B(ID);  
ROLLBACK;  
SELECT * FROM A
```

Below the script editor, the status bar indicates "Task completed in 0.049 seconds". The main window displays an error report for the command "CREATE INDEX IDX_T on B(ID)".

Error starting at line : 10 in command -
CREATE INDEX IDX_T on B(ID)
Error report -
ORA-00955: name is already used by an existing object
00955. 00000 - "name is already used by an existing object"
*Cause:
*Action:

Handwritten orange notes are present on the error report:

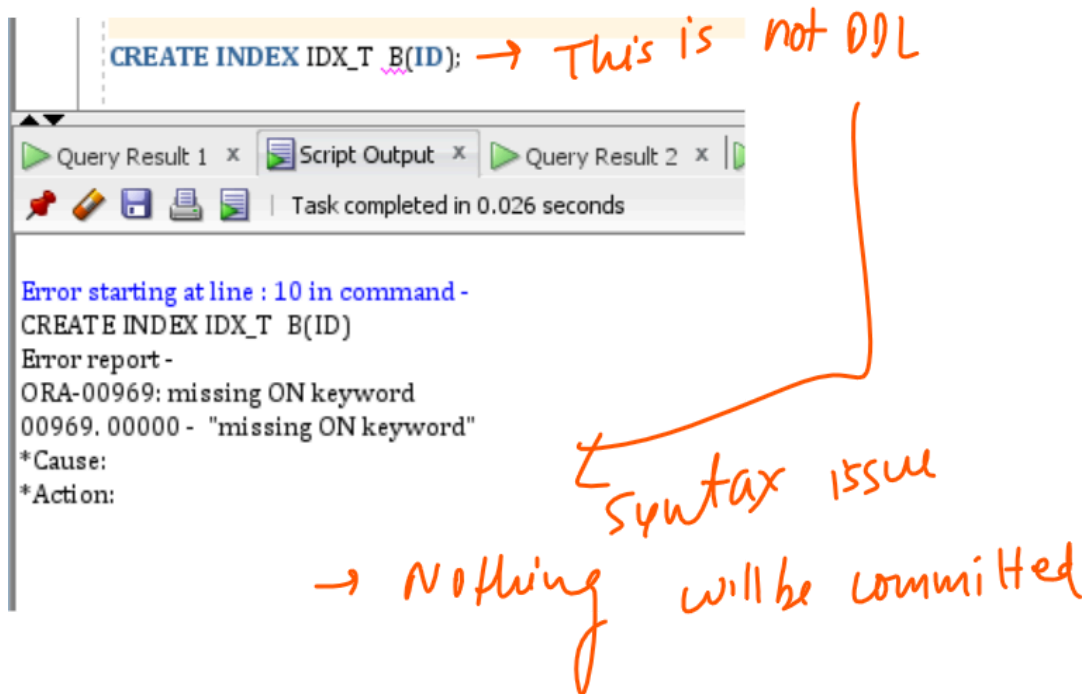
- An arrow points from "ORA-00955" to "Consider DDL & commit data".
- A bracket on the right side of the error report is labeled "DDL fired" with an arrow pointing down to "not executed" and "boz name exist".

You can see in above example, DDL failed, but still data was inserted in table. Because the concept of DDL is first fire commit internally and then run itself. This question will be twist in interview , when they said, what happened if DDL fails. Remember, if DDL fails (Other than syntax issue) , it commits.

Scenario 5:

```
/* 4 rows Inserted */  
INSERT INTO A VALUES (1);  
INSERT INTO A VALUES (2);  
INSERT INTO A VALUES (3);  
INSERT INTO A VALUES (4);
```

After that I issued a DDL command. but my ddl fail, but it should be syntax error. SYNTAX error means, database not consider that you are a DDL. data will not commit.



But If DDL fails because of Syntax issue, means database does not consider it as DDL, it will not commit.

Datatypes:

```
create table employee(
EMPID number,
EMPNAME varchar2(10),
age number,
sex char(4)
);
```

varchar is for string.
number is for number.

varchar2(10) --> 10 is the capacity of that column. it can maimum store the 10 character string.

--> syntax --> means how the programming language talk to system.

--> **Insert syntax**

--> Insert Into <tablename> Values(the value you want to insert);

```
insert into employee values (1,'hsgsggs',100,'M');
```

select * from employee; * means we want to see all data on the table.

If I am trying to insert values more than 10 character like:

```
insert into employee values (1,'hsgsyeyyeetggs',100,'M');
```

It gives me an error, because the capacity of the empname column is 10 and we are trying to insert more than 10 character strings.

varchar2(10) --> indicates the total capacity of this column is 10. we can not insert any data which length is more than 10

--> In varchar2 columns we can insert number values , but in number columns we can not insert varchar2 values

```
insert into emp values (2, 44, 40, 'M')
```

1 row(s) inserted.

```
SELECT * FROM EMP WHERE empname = '44'
```

EMPID	EMPNAME	AGE	SEX
2	44	40	M

↳ Number inserted into varchar

```
SELECT * FROM EMP WHERE empname = '44'
```

EMPID	EMPNAME	AGE	SEX
2	44	40	M

↳ Treated as varchar
↓

once you come to my family, you have to follow my custom

=====

```
create table xyz (name varchar2(20), active char(20))
```

Table created.

```
insert into xyz values ('abcd', 'Y')
```

1 row(s) inserted.

```
select name, length(name), active, length(active) from xyz
```

NAME	LENGTH(NAME)	ACTIVE	LENGTH(ACTIVE)
abcd	4	Y	20

assigned 20 but use ↑

use all 20

Difference between char and varchar ?

The short answer is: VARCHAR is variable length, while CHAR is fixed length. CHAR is a fixed length string data type, so any remaining space in the field is padded with blanks. CHAR takes up 1 byte per character. ... VARCHAR is a variable length string data type, so it holds only the characters you assign to it.

Prefer to use varchar2

Difference between char, varchar and VARCHAR2 in Oracle :

Sno	Char	VarChar/VarChar2
1	Char stands for “Character”	VarChar/VarChar2 stands for Variable Character
2	It is used to store character string of fixed length	It is used to store character string of variable length
3	It has a Maximum Size of 2000 Bytes	It has a Maximum Size of 4000 Bytes

Sno	Char	VarChar/VarChar2
4	Char will pad the spaces to the right side to fill the length specified during the Declaration	VarChar will not pad the spaces to the right side to fill the length specified during Declaration.
6	It is Static Datatype(i.e Fixed Length)	It is Dynamic Datatype(i.e Variable Length)
7	It can lead to memory wastage	It manages Memory efficiently
8	It is 50% much faster than VarChar/VarChar2	It is relatively slower as compared to Char

Note : There is no difference between VarChar and VarChar2 in Oracle. However, it is advised not to use VarChar for storing data as it is reserved for future use for storing some other type of variable. Hence, always use VarChar2 in place of VarChar.

Hence it is advised to use Char datatype when the length of the character string is fixed and will not change in the future.

If the length of the character string is not fixed, then VarChar2 is preferred

|